User Stories & Gherkin Acceptance Criteria — WariMitra

Document Identifier: WM-US-009

Classification: Agile Epics, User Stories & Behavioral Acceptance Tests

Standard: INVEST Criteria & Gherkin Syntax (Given-When-Then)

Target Audience: Scrum Teams, QA Automation Engineers, Business Analysts, Agile Interviewers

Source Evidence: `functions/src/index.ts`, `frontend/src/app/`, `mobile/lib/screens/`

1. Epic Overview

Epic ID	Epic Name	Strategic Goal
EPC-01	Emergency SOS & Crisis Response	Deliver sub-2-second panic alerts with precise GPS coordinates directly to police and medical units.
EPC-02	Geospatial Healthcare & Medical Relief	Enable pilgrims to locate the nearest active medical camp and manage clinic bed occupancy.
EPC-03	Command Center & Incident Triage	Provide emergency dispatchers with a real-time, virtualized operational view over all field incidents.
EPC-04	Family Circle & Missing Person Recovery	Prevent family separation and streamline missing person intake and verification.
EPC-05	Temple Queue & Crowd Telemetry	Provide live wait-time visibility and slot booking to eliminate crowd crushing.
EPC-06	Web3 Trust & Governance	Maintain an immutable cryptographic audit ledger of high-priority emergency interventions.

2. Detailed User Story Inventory

Epic 1: Emergency SOS & Crisis Response

US-SOS-001: 3-Second Hold-to-Trigger Panic Button

- **Epic:** EPC-01 (Emergency SOS)
- **User Story:**

> **As a** walking pilgrim experiencing acute chest pain or dehydration,
> **I want** to hold down a prominent red SOS button on my phone for 3 seconds,
> **So that** emergency services are alerted immediately without me needing to type coordinates or navigate complex menus.

- **Business Value:** Prevents accidental false alarms while ensuring guaranteed dispatch within the clinical "Golden Hour".
- **Priority:** Must Have
- **Dependencies:** Device GPS hardware, Firebase Auth.
- **Acceptance Criteria (Gherkin):**

```gherkin
Scenario: Successful panic trigger after 3-second continuous press
Given the pilgrim has the SOS screen open on an active GPS-enabled device
When the pilgrim presses and holds the circular SOS button continuously for 3.0 seconds
Then the button shall display an animated circular progress ring and haptic vibration
And at exactly 3.0 seconds, the createSOS Cloud Function shall be invoked with the device coordinates
And the screen shall transition to the active dispatch status with direct emergency dial options

Scenario: Early release cancels panic trigger
Given the pilgrim begins pressing the SOS button
When the pilgrim releases their finger after 1.8 seconds
Then the circular progress ring shall reset to zero
And no network request or Cloud Function invocation shall be made
```

US-SOS-002: Idempotent Emergency Request Ingestion

- **Epic:** EPC-01 (Emergency SOS)
- **User Story:**

> **As a** mobile client operating on a fluctuating rural 3G/4G network,
> **I want** the backend to recognize my client-generated idempotency key,
> **So that** automatic network retries do not spawn duplicate emergency tickets or waste police patrol

So that automatic network retries do not spawn duplicate emergency tickets or waste police patrol resources.

- **Business Value:** Eliminates duplicate dispatches and protects command center operators from alert fatigue.
- **Priority:** Must Have
- **Dependencies:** US-SOS-001.
- **Acceptance Criteria (Gherkin):**

```
gherkin Scenario: Duplicate packet arrives due to network timeout retry Given an SOS alert already exists in Firestore with idempotencyKey "uuid-9876-wxyz" When the client retransmits createSOS with the identical idempotencyKey "uuid-9876-wxyz" Then the function shall NOT insert a second document into the sosAlerts collection And the function shall return the existing alertId with message "SOS already exists (duplicate prevented)"
```

US-SOS-003: Rate Limiting Abuse Prevention

- **Epic:** EPC-01 (Emergency SOS)
- **User Story:**

As a system administrator,

I want the backend to cap emergency SOS creation to 5 alerts per minute per user account,

So that malicious actors or malfunctioning bots cannot flood the database or denial-of-service the responders.

- **Business Value:** Ensures system resilience against denial-of-service attacks.
- **Priority:** Must Have
- **Dependencies:** Firebase Authentication.
- **Acceptance Criteria (Gherkin):**

```
gherkin Scenario: User attempts to exceed 5 alerts within 60 seconds Given a user account that has successfully created 5 alerts in the last 50 seconds When the user submits a 6th createSOS call Then the function shall abort with HttpsError status "resource-exhausted" And no new document shall be added to the sosAlerts collection
```

Epic 2: Geospatial Healthcare & Medical Relief

US-MED-001: Finding Nearest Medical Camp via Geohash Radar

- **Epic:** EPC-02 (Geospatial Healthcare)

- **User Story:**

> **As a** ground volunteer assisting an injured pilgrim,
>
> **I want** to see an ordered list of active medical camps within 10 km sorted by walking distance,
>
> **So that** I can direct the pilgrim to the closest medical tent without making unnecessary phone inquiries.

- **Business Value:** Reduces time-to-treatment by up to 80% and decentralizes minor trauma treatment.
- **Priority:** Must Have
- **Dependencies:** Device GPS, `geofire-common`.
- **Acceptance Criteria (Gherkin):**

```
gherkin Scenario: Querying nearby camps with default 10 km radius Given the user is at latitude 18.5204, longitude 73.8567 And there are 3 active medical camps at distances 1.2 km, 4.8 km, and 12.5 km When the user calls the getMedicalCamps function Then the response shall return the 1.2 km and 4.8 km camps in ascending distance order And the 12.5 km camp shall be omitted from the results
```

US-MED-002: Medical Camp Bed Capacity Telemetry

- **Epic:** EPC-02 (Geospatial Healthcare)
- **User Story:**

> **As an** on-duty Medical Officer at an emergency field tent,
>
> **I want** to update our current patient headcount and bed capacity,
>
> **So that** incoming ambulances and nearby volunteers can see our real-time availability before dispatching critical patients to our clinic.

- **Business Value:** Prevents sudden clinic bed exhaustion during mass heatstroke outbreaks.
- **Priority:** Must Have
- **Dependencies:** RBAC custom claim `medical_officer`.
- **Acceptance Criteria (Gherkin):**

```
gherkin Scenario: Medical officer updates bed count Given an authenticated user with role "medical_officer" When the officer updates currentPatients to 18 on a camp with total capacity 20 Then the Firestore document /medicalCamps/{campId} shall be updated immediately And the client radar shall reflect "2 beds available"
```

Epic 3: Command Center & Incident Triage

US-CMD-001: Virtualized Real-Time Emergency Incident Feed

- **Epic:** EPC-03 (Command Center)
- **User Story:**

> **As a** police control room operator,
> **I want** the web emergency feed to stream incoming SOS alerts in real time using list virtualization,
> **So that** my browser remains responsive and stutter-free even when hundreds of incidents are logged simultaneously.

- **Business Value:** Zero software crash during peak crowd surges; 100% incident capture.
- **Priority:** Must Have
- **Dependencies:** Next.js 15, `react-window`, Firestore listeners.
- **Acceptance Criteria (Gherkin):**

```gherkin
Scenario: Streaming 500 active incidents smoothly
Given the web dashboard has 500 pending alerts in the incident store
When the operator scrolls through the emergency queue
Then react-window shall render only the currently visible DOM nodes
And scrolling shall maintain 60 frames per second without memory accumulation
```

Epic 4: Family Circle & Missing Person Recovery

US-FAM-001: 6-Character Invite Code Family Circle

- **Epic:** EPC-04 (Family & Missing Persons)
- **User Story:**

> **As a** pilgrim walking with my family,
> **I want** to generate a simple 6-character code that my elderly parents can enter on their phones,
> **So that** we can view each other's live locations on a shared map without needing complex social media accounts.

- **Business Value:** Reduces family separation incidents by over 70%.
- **Priority:** Should Have
- **Dependencies:** Firebase Firestore.
- **Acceptance Criteria (Gherkin):**

```gherkin
Scenario: Creating and joining a family circle
Given a pilgrim creates a new
```

- ```
  gherkin Scenario: Creating and joining a family circle Given a pilgrim creates a new family group named "Patil Family" When the system generates a unique 6-character code "WARI99" And another family member enters "WARI99" in their app Then their userId shall be added to the memberIds array of the family group And both members shall be able to view each other's location pins on the map
  ```

Epic 5: Web3 Trust & Governance

US-W3-001: Tamper-Evident Blockchain Proof Anchoring

- **Epic:** EPC-06 (Web3 Trust)
- **User Story:**

  > **As a** District Collector or Judicial Commission Officer,
  > **I want** every emergency SOS state change to be cryptographically hashed and anchored to a public testnet blockchain,
  > **So that** administrative response times can be audited post-event with non-repudiable proof that records were never doctored.

- **Business Value:** Absolute public transparency and statutory protection against false negligence lawsuits.
- **Priority:** Should Have
- **Dependencies:** Ethers.js v6, Sepolia Testnet, Cloud Functions.
- **Acceptance Criteria (Gherkin):**

  ```
  gherkin Scenario: Asynchronous anchoring of SOS status update Given an existing SOS alert transitions from "pending" to "dispatched" When the onSOSUpdate Cloud Function triggers Then it shall create a canonical JSON snapshot containing entityId, new status, and timestamp And it shall compute a SHA-256 hash of the snapshot And it shall anchor the hash via an async 0-ETH transaction to Sepolia And it shall record the transactionHash in the blockchainProofs collection
  ```
  ```